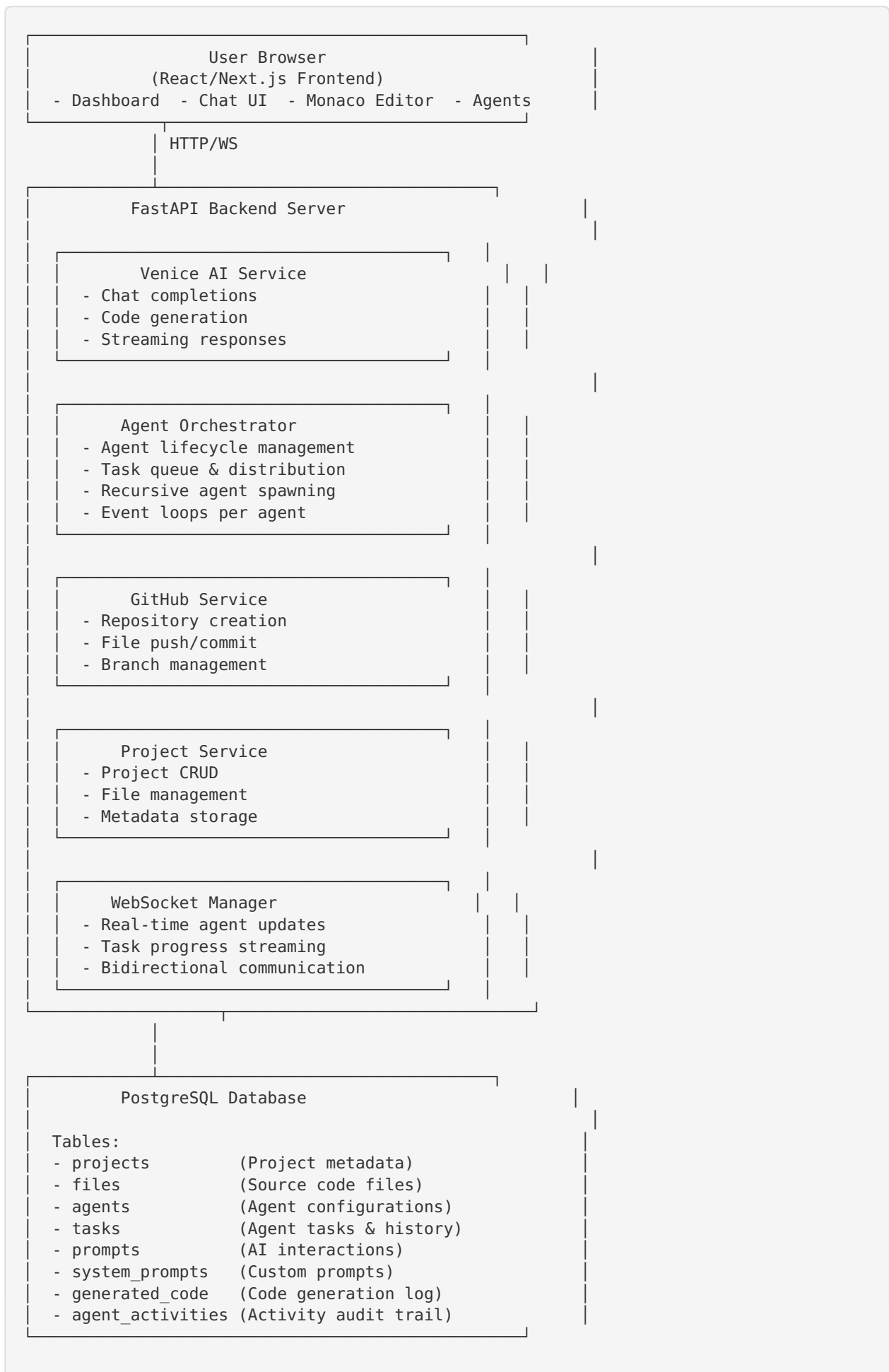


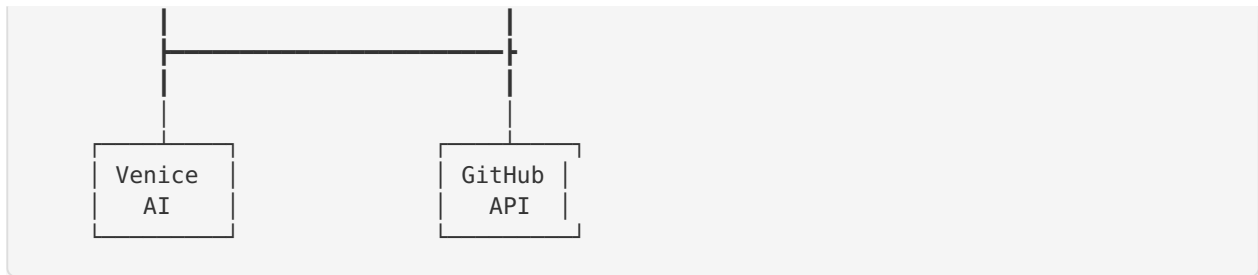
CodeForge AI - Architecture Documentation

System Overview

CodeForge AI is a local web-based AI development platform that leverages Venice AI's Dolphin uncensored model for code generation, features multi-agent orchestration, and integrates with GitHub for version control.

Architecture Diagram





Key Components

1. Frontend (Next.js/React)

Purpose: User interface for interacting with the system

Key Features:

- **Dashboard:** Overview of projects, agents, and activity
- **Chat Interface:** Real-time AI conversation with streaming responses
- **Monaco Editor:** Full-featured code editor with syntax highlighting
- **Agent Panel:** Manage and monitor AI agents
- **Project Workspace:** Browse and edit project files

Technology Stack:

- Next.js 14 (App Router)
- TypeScript
- Tailwind CSS
- Zustand (State Management)
- TanStack Query (Data Fetching)
- Socket.io-client (WebSocket)
- Monaco Editor

2. Backend (FastAPI)

Purpose: Core application logic and API endpoints

Services:

Venice AI Service

- Handles all AI model interactions
- Streaming response support
- Custom system prompt management
- Code block extraction and parsing

Agent Orchestrator

- Creates and manages AI agents
- Maintains agent lifecycle (active, idle, terminated)
- Task queue management
- Recursive agent spawning
- Event loop per agent
- Parent-child agent relationships

GitHub Service

- Repository creation

- File push/commit operations
- Branch management
- Repository status tracking

Project Service

- Project CRUD operations
- File management
- Language detection
- Metadata storage

WebSocket Manager

- Real-time bidirectional communication
- Client connection management
- Topic-based subscriptions
- Broadcast and unicast messaging

3. Database (PostgreSQL)

Schema:

```

-- Projects
projects (
  id UUID PRIMARY KEY,
  name VARCHAR(255),
  description TEXT,
  framework VARCHAR(100),
  status VARCHAR(50),
  created_at TIMESTAMP,
  updated_at TIMESTAMP
)

-- Files
files (
  id UUID PRIMARY KEY,
  project_id UUID REFERENCES projects(id),
  path VARCHAR(500),
  content TEXT,
  language VARCHAR(50),
  created_at TIMESTAMP,
  updated_at TIMESTAMP
)

-- Agents
agents (
  id VARCHAR PRIMARY KEY,
  name VARCHAR(255),
  role VARCHAR(100),
  capabilities JSON,
  system_prompt TEXT,
  parent_id VARCHAR REFERENCES agents(id),
  status VARCHAR(50),
  created_at TIMESTAMP,
  terminated_at TIMESTAMP
)

-- Tasks
tasks (
  id VARCHAR PRIMARY KEY,
  agent_id VARCHAR REFERENCES agents(id),
  type VARCHAR(100),
  description TEXT,
  data JSON,
  status VARCHAR(50),
  result JSON,
  error TEXT,
  created_at TIMESTAMP,
  completed_at TIMESTAMP
)

-- Prompts
prompts (
  id UUID PRIMARY KEY,
  project_id UUID REFERENCES projects(id),
  prompt_text TEXT,
  response_text TEXT,
  model_used VARCHAR(100),
  tokens_used INTEGER,
  created_at TIMESTAMP
)

-- System Prompts
system_prompts (

```

```

    id UUID PRIMARY KEY,
    name VARCHAR(255),
    content TEXT,
    category VARCHAR(100),
    is_active BOOLEAN,
    created_at TIMESTAMP,
    updated_at TIMESTAMP
)

-- Generated Code
generated_code (
    id VARCHAR PRIMARY KEY,
    raw_response TEXT,
    code_blocks JSON,
    language VARCHAR(50),
    timestamp VARCHAR(100),
    created_at TIMESTAMP
)

-- Agent Activities
agent_activities (
    id UUID PRIMARY KEY,
    agent_id VARCHAR REFERENCES agents(id),
    activity_type VARCHAR(100),
    description TEXT,
    metadata JSON,
    timestamp TIMESTAMP
)

```

Data Flow

1. Code Generation Flow

```

User Input (Chat)
  ↓
Frontend validates
  ↓
POST /api/chat
  ↓
Venice AI Service
  ↓
Stream response chunks
  ↓
Frontend displays (ReactMarkdown)
  ↓
Store in database (Prompts table)

```

2. Agent Task Flow

```

User creates agent
↓
AgentOrchestrator.create_agent()
↓
Agent instance created
↓
Event loop started (asyncio.Task)
↓
Task assigned to agent
↓
Task added to agent's queue
↓
Event loop processes task
↓
Venice AI generates response
↓
Result stored in database
↓
WebSocket broadcasts update
↓
Frontend updates UI

```

3. GitHub Push Flow

```

User selects files
↓
POST /api/github/push
↓
GitHub Service
↓
Create blobs for each file
↓
Create tree object
↓
Create commit
↓
Update branch reference
↓
Return commit SHA
↓
Frontend shows success

```

Security Considerations

1. API Key Management

- All API keys stored in `.env` files (never committed)
- Environment variables loaded via `python-dotenv`
- Frontend has no access to backend credentials

2. CORS Configuration

- Restricted to `localhost:3000` in development
- Should be configured for production domain

3. Database Security

- Password-protected PostgreSQL user
- Connection string in environment variables
- Prepared statements via SQLAlchemy (SQL injection prevention)

4. Input Validation

- Pydantic models for request validation
- Type checking on all endpoints
- SQL injection prevention via ORM

Performance Optimizations

1. Async Operations

- FastAPI with async/await
- Async database queries (asyncpg)
- Non-blocking AI API calls

2. Streaming Responses

- Server-Sent Events (SSE) for AI responses
- Reduces time to first token
- Better user experience

3. Connection Pooling

- PostgreSQL connection pool via SQLAlchemy
- Reuses database connections
- Reduces connection overhead

4. Frontend Optimizations

- React Query for data caching
- Zustand for minimal re-renders
- Code splitting with Next.js
- Monaco Editor lazy loading

Scalability

Current Architecture (Single Machine)

- Suitable for: 1-10 concurrent users
- Performance: Fast for local development
- Limitations: Single point of failure

Future Scaling Options

1. Horizontal Scaling:

- Multiple FastAPI instances behind load balancer
- Shared PostgreSQL database
- Redis for session storage and WebSocket pub/sub

- 2. **Vertical Scaling:**
 - Increase machine resources
 - More CPU cores for parallel agent processing
 - More RAM for caching
- 3. **Database Scaling:**
 - Read replicas for queries
 - Master-slave replication
 - Connection pooling
- 4. **Agent Distribution:**
 - Task queue (Celery, RabbitMQ)
 - Distributed agent workers
 - Load balancing across workers

Deployment Architecture

Development (Current)

Localhost:3000 (Frontend)
↓
Localhost:8000 (Backend)
↓
Localhost:5432 (PostgreSQL)

Production (Recommended)



Monitoring & Observability

Recommended Tools

- 1. **Logging:** Python `logging` module + file rotation
- 2. **Metrics:** Prometheus + Grafana
- 3. **Tracing:** OpenTelemetry
- 4. **Error Tracking:** Sentry
- 5. **Uptime Monitoring:** UptimeRobot

Key Metrics to Track

- API response times
- Agent task completion rates
- Venice AI API latency

- Database query performance
- WebSocket connection count
- Error rates per endpoint

Future Enhancements

1. **Authentication & Authorization**

- User accounts
- Role-based access control
- API key management

2. **Collaborative Features**

- Real-time collaboration on projects
- Shared agents across users
- Team workspaces

3. **Advanced Agent Features**

- Agent templates
- Custom tool integration
- Agent marketplace

4. **Cloud Integration**

- AWS/GCP/Azure deployment
- Serverless functions
- Container orchestration (Kubernetes)

5. **AI Model Options**

- Multiple AI provider support
- Local model integration (Ollama)
- Model switching per agent

For implementation details, see source code in `/backend` and `/frontend` directories.